# Research Report 01:
## Picture-in-Picture Sign Language Interpreter Detection Development and Optimization

Voice-of-Hands Research Team

April 15, 2026

**Abstract**

This report documents the development, testing, and optimization of the Picture-in-Picture (PiP) sign language interpreter detection system for the Voice-of-Hands automated dataset creation pipeline. We present four detection methods—HSV color-space border analysis, Farneback optical flow, Canny edge detection, and MediaPipe pose estimation—and their integration into a multi-method cascade approach achieving 92% detection accuracy on Sri Lankan parliamentary broadcast footage.

## Contents

# 1 Introduction

## 1.1 Problem Statement

Sri Lankan parliamentary broadcasts embed professional sign language interpreters in a Picture-in-Picture (PiP) overlay, typically positioned in the bottom-right corner of the video frame. Automatic detection and localization of this PiP region is the critical first stage of our multimodal dataset creation pipeline, as all subsequent processing (cropping, activity detection, segmentation) depends on accurate interpreter localization.

## 1.2 Challenges

1. **Variable Positioning**: While interpreters are typically in the bottom-right corner, the exact position varies across different broadcast sessions and even within a single session.
2. **Background Complexity**: Parliamentary proceedings include moving backgrounds, changing speakers, and visual transitions that can interfere with detection algorithms.
3. **Border Variability**: The border frame surrounding the PiP overlay varies in color (white, light gray, beige), thickness (2–8 pixels), and transparency across different broadcasts.
4. **Lighting Conditions**: Broadcast lighting conditions change throughout sessions, affecting color-based detection methods.
5. **Computational Efficiency**: Detection must complete within seconds to maintain pipeline throughput, especially when processing multi-hour broadcasts.
6. **Robustness Requirements**: The system must achieve $> 90\%$ detection accuracy to avoid requiring manual intervention.

## 1.3 Research Objectives

- Develop multiple complementary detection methods
- Optimize parameters for Sri Lankan broadcast conditions
- Achieve detection accuracy $> 90\%$
- Minimize detection time ($< 5$ seconds per video)
- Create a robust cascade system that selects the best method per video

# 2 Method 1: HSV Color-Space Border Detection

## 2.1 Theoretical Foundation

The HSV (Hue, Saturation, Value) color space separates chromatic content (hue, saturation) from illumination (value), providing robustness to lighting variations compared to RGB space. Light-colored PiP borders—typically white, cream, or light gray—exhibit distinct HSV characteristics:

- **High Value (V)**: Bright colors have $V > 180$ (on 0–255 scale)
- **Low Saturation (S)**: Achromatic colors have $S < 80$
- **Unrestricted Hue (H)**: White/gray borders span all hue values

## 2.2 Algorithm Design

### 2.2.1 Color Space Transformation

Given a BGR frame $\mathbf{I}_{\text{BGR}} \in \mathbb{R}^{H \times W \times 3}$, we apply:

$$\mathbf{I}_{\text{HSV}} = f_{\text{BGR}\to\text{HSV}}(\mathbf{I}_{\text{BGR}}) \tag{1}$$

using OpenCV's `cv2.cvtColor()` function.

### 2.2.2 Color Thresholding

A binary mask isolates light-colored pixels:

$$M_{\text{light}}(x, y) = \begin{cases} 1 & \text{if } H \in [0, 180] \wedge S \in [0, 80] \wedge V \in [180, 255] \\ 0 & \text{otherwise} \end{cases} \tag{2}$$

**Implementation:**

```
lower_light = np.array([0, 0, 180])
upper_light = np.array([180, 80, 255])
mask = cv2.inRange(hsv_frame, lower_light, upper_light)
```

### 2.2.3 Morphological Refinement

The raw binary mask contains noise and gaps. We apply:

$$M_{\text{closed}} = \phi_{\text{close}}(M_{\text{light}}, K_{3\times3}) \tag{3}$$

$$M_{\text{refined}} = \phi_{\text{open}}(M_{\text{closed}}, K_{3\times3}) \tag{4}$$

where $K_{3\times3}$ is a rectangular structuring element.
**Purpose:**

- **Closing** ($\phi_{\text{close}}$): Fills small gaps in border contours
- **Opening** ($\phi_{\text{open}}$): Removes isolated noise pixels

### 2.2.4 Contour Extraction and Filtering

Contours are extracted using:

```
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
                               cv2.CHAIN_APPROX_SIMPLE)
```

Each contour is filtered based on:

1. **Area constraint**:
$$A_{\text{min}} < A_{\text{contour}} < A_{\text{max}} \tag{5}$$
where $A_{\text{min}} = 0.008 \cdot W \cdot H$ and $A_{\text{max}} = 0.25 \cdot W \cdot H$.

2. **Aspect ratio constraint**:
$$0.5 < \frac{w}{h} < 2.0 \tag{6}$$
ensuring roughly square or slightly rectangular shapes.

3. **Corner region constraint**: Bounding box center must lie in outer 45% of frame:
$$(x_c > 0.55W) \vee (x_c < 0.45W) \quad \text{(horizontal)} \tag{7}$$
$$(y_c > 0.55H) \vee (y_c < 0.45H) \quad \text{(vertical)} \tag{8}$$

#### 2.2.5 Interior Validation

To distinguish true PiP borders from false positives (e.g., white text boxes, graphics), we verify that:

1. **Interior is dark** (contains interpreter against dark studio background)
2. **Border is bright** (light-colored frame)

The interior region excludes a 15% margin:

$$\Omega_{\text{int}} = \{(x, y) : x_1 + 0.15w < x < x_2 - 0.15w,\ y_1 + 0.15h < y < y_2 - 0.15h\} \tag{9}$$

Validation conditions:

$$\mu_{\text{interior}} = \frac{1}{|\Omega_{\text{int}}|} \sum_{(x,y)\in\Omega_{\text{int}}} V(x, y) < 100 \tag{10}$$

$$\mu_{\text{border}} = \frac{1}{|\Omega_{\text{border}}|} \sum_{(x,y)\in\Omega_{\text{border}}} V(x, y) > 180 \tag{11}$$

### 2.3 Parameter Optimization

Table 1: HSV Border Detection Parameter Optimization

| **Parameter** | **Initial** | **Optimized** | **Justification** |
|---|---|---|---|
| Saturation max | 50 | 80 | Handle cream/beige borders |
| Value min | 200 | 180 | Handle dimmer borders |
| Interior margin | 0.10 | 0.15 | Avoid border pixels in interior |
| Min area | 0.005 | 0.008 | Filter small false positives |
| Max area | 0.30 | 0.25 | Ensure PiP size constraint |
| Aspect ratio | [0.7, 1.5] | [0.5, 2.0] | Handle slight rectangles |

### 2.4 Performance Results

Table 2: HSV Border Detection Performance

| **Metric** | **Value** |
|---|---|
| Detection accuracy | 92% |
| Average detection time | 3.2 seconds |
| False positive rate | 3% |
| False negative rate | 8% |
| Confidence score (avg) | 0.85 |
| Frames sampled | 30 frames/video |

### 2.5 Failure Modes

1. **No visible border**: Some broadcasts use borderless PiP overlays (8% of cases)
2. **Colored borders**: Blue or dark-colored borders fall outside HSV thresholds (rare, $< 1\%$)

3. **High interior brightness**: When interpreter wears white clothing or bright background is used (2%)

4. **Multiple light regions**: Overlapping text graphics or scoreboards (3%)

# 3 Method 2: Farneback Dense Optical Flow

## 3.1 Motivation

When HSV border detection fails ($c_{\text{border}} < 0.25$), motion-based detection provides a robust fallback. Sign language interpreters exhibit continuous hand and arm motion during active signing, creating a distinct motion signature compared to the relatively static parliamentary background.

## 3.2 Theoretical Background

The Farneback optical flow algorithm [**?**] approximates each frame neighborhood with a quadratic polynomial:

$$f(x) \approx x^T A x + b^T x + c \tag{12}$$

Displacement between frames is computed by polynomial expansion matching, providing dense per-pixel motion vectors $\mathbf{u} = (u_x, u_y)$.

## 3.3 Implementation

### 3.3.1 Flow Computation

```
flow = cv2.calcOpticalFlowFarneback(
    prev_gray, curr_gray,
    None,               # Output flow (computed)
    pyr_scale=0.5,      # Pyramid scale factor
    levels=3,           # Pyramid levels
    winsize=15,         # Averaging window size
    iterations=3,       # Iterations per pyramid level
    poly_n=5,           # Polynomial expansion neighborhood
    poly_sigma=1.2,     # Gaussian std for polynomial expansion
    flags=0
)
```

### 3.3.2 Motion Magnitude Heatmap

Per-pixel motion magnitude:

$$M(x, y) = \sqrt{u_x(x, y)^2 + u_y(x, y)^2} \tag{13}$$

Accumulated across $N = 30$ sampled frame pairs:

$$H(x, y) = \sum_{t=1}^{N} M_t(x, y) \tag{14}$$

Normalized heatmap:

$$\hat{H}(x, y) = \frac{H(x, y)}{\max_{x', y'} H(x', y')} \times 255 \tag{15}$$

#### 3.3.3 Peak Detection

The interpreter region corresponds to the maximum motion peak within corner ROIs:

```
# Apply corner region mask
heatmap_corners = heatmap * corner_mask

# Find peak location
(min_val, max_val, min_loc, max_loc) = cv2.minMaxLoc(heatmap_corners)

# Extract region around peak
x, y = max_loc
bbox = (x - w/2, y - h/2, x + w/2, y + h/2)
```

### 3.4 Parameter Tuning

Table 3: Optical Flow Parameter Optimization

| Parameter | Initial | Final | Impact |
|---|---|---|---|
| Pyramid scale | 0.7 | 0.5 | Faster processing |
| Pyramid levels | 5 | 3 | Reduced blur |
| Window size | 21 | 15 | Better localization |
| Frames sampled | 50 | 30 | Speed improvement |
| Motion threshold | 150 | 120 | Higher sensitivity |

### 3.5 Performance

Table 4: Optical Flow Detection Performance

| Metric | Value |
|---|---|
| Detection accuracy | 78% |
| Average detection time | 10.5 seconds |
| Confidence score (avg) | 0.65 |
| Success on HSV failures | 65% |

### 3.6 Limitations

1. **Computational cost**: 3× slower than HSV method
2. **Camera motion sensitivity**: Panning or zooming cameras create global motion artifacts
3. **Background activity**: Moving speakers or audience members can interfere
4. **Idle periods**: When interpreter is resting, motion magnitude drops

## 4 Method 3: Canny Edge Detection

### 4.1 Concept

Canny edge detection identifies strong intensity gradients characteristic of PiP border edges. The Canny algorithm applies:

1. Gaussian smoothing
2. Gradient magnitude and direction computation
3. Non-maximum suppression
4. Hysteresis thresholding

### 4.2 Implementation

```
# Grayscale conversion
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

# Gaussian blur to reduce noise
blurred = cv2.GaussianBlur(gray, (5, 5), 1.4)

# Canny edge detection
edges = cv2.Canny(blurred,
                  threshold1=50,   # Lower threshold
                  threshold2=150,  # Upper threshold
                  apertureSize=3)

# Find contours
contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL,
                               cv2.CHAIN_APPROX_SIMPLE)
```

### 4.3 Rectangle Filtering

Detected contours are approximated as polygons:

$$\text{poly} = \text{approxPolyDP}(\text{contour}, \epsilon \cdot \text{perimeter}, \text{closed} = \text{True}) \tag{16}$$

where $\epsilon = 0.04$ controls approximation precision.
Rectangles are identified by:

$$|\text{vertices}| = 4 \quad \wedge \quad \text{isContourConvex}(\text{poly}) \tag{17}$$

### 4.4 Performance

Table 5: Canny Edge Detection Performance

| **Metric** | **Value** |
|---|---|
| Detection accuracy | 71% |
| Detection time | 5.8 seconds |
| Confidence score | 0.55 |
| False positives | High (15%) |

### 4.5 Challenges

- **High false positives**: Text boxes, graphics, and furniture edges detected
- **Incomplete rectangles**: Border gaps lead to broken contours
- **Background complexity**: Parliamentary chamber contains many rectangular structures

# 5 Method 4: MediaPipe Pose Estimation

## 5.1 Approach

MediaPipe Pose detects 33 body landmarks (shoulders, elbows, wrists, etc.) using the BlazePose neural network architecture. For PiP detection, we:

1. Run MediaPipe Pose on corner ROIs
2. Identify regions containing upper-body landmarks
3. Compute bounding box from detected keypoints

## 5.2 Implementation

```
import mediapipe as mp

mp_pose = mp.solutions.pose
pose = mp_pose.Pose(
    static_image_mode=False,
    min_detection_confidence=0.5,
    min_tracking_confidence=0.5
)

# Process frame
results = pose.process(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))

if results.pose_landmarks:
    # Extract upper body landmarks (11-16: shoulders, elbows, wrists)
    upper_body_points = [results.pose_landmarks.landmark[i]
                         for i in range(11, 17)]

    # Compute bounding box
    xs = [p.x * width for p in upper_body_points]
    ys = [p.y * height for p in upper_body_points]
    bbox = (min(xs), min(ys), max(xs), max(ys))
```

## 5.3 Performance

Table 6: MediaPipe Pose Detection Performance

| Metric | Value |
|---|---|
| Detection accuracy | 68% |
| Detection time | 12.3 seconds |
| Confidence score | 0.60 |
| GPU acceleration | Yes (CUDA) |

## 5.4 Limitations

- **Slow inference**: Neural network requires GPU for real-time performance
- **Scale dependency**: Small interpreters ($< 100$ pixels) yield low confidence
- **Occlusion sensitivity**: Partial visibility reduces detection rate
- **Background detections**: May detect speakers in main frame

# 6 Multi-Method Cascade Integration

## 6.1 Design Philosophy

No single detection method achieves 100% accuracy across all broadcast conditions. Our cascade approach:

1. Tries fast, high-accuracy methods first (HSV border)
2. Falls back to robust alternatives when confidence is low
3. Selects the result with highest confidence score
4. Provides graceful degradation

## 6.2 Cascade Logic

$$\text{Method} = \begin{cases} \text{Border} & \text{if } c_{\text{border}} > 0.25 \\ \text{Motion} & \text{if } c_{\text{motion}} > 0.60 \\ \arg\max(c_{\text{edge}}, c_{\text{motion}}) & \text{otherwise} \end{cases} \tag{18}$$

## 6.3 Confidence Scoring

Each method computes a confidence score $c \in [0, 1]$ based on:

- **Border**: Interior/border contrast ratio
- **Motion**: Peak motion magnitude
- **Edge**: Contour completeness and rectangularity
- **Pose**: Landmark detection confidence

## 6.4 Combined Performance

Table 7: Cascade System Performance

| Metric | Individual Best | Cascade |
|---|---|---|
| Detection accuracy | 92% (Border) | **96%** |
| Avg detection time | 3.2s (Border) | **4.1s** |
| False negative rate | 8% | **4%** |
| Robustness score | 0.85 | **0.92** |

# 7 Problems Encountered and Solutions

## 7.1 Problem 1: Border Color Variability

**Issue**: Initial implementation used fixed HSV thresholds optimized for pure white borders, failing on cream or light gray borders observed in 15% of videos.

**Solution**:

- Expanded saturation range from [0, 50] to [0, 80]
- Lowered value threshold from 200 to 180

- Increased tolerance to slight color tints

**Result**: Detection rate improved from 77% to 92%.

### 7.2 Problem 2: Interior Validation False Negatives

**Issue**: When interpreters wore bright white shirts or studio had bright backgrounds, the interior brightness check ($\mu_{\text{int}} < 100$) rejected valid detections.

**Solution**:

- Added adaptive threshold based on border-interior contrast ratio
- Relaxed absolute threshold to 100 (from initial 80)
- Weighted interior center more than edges

### 7.3 Problem 3: Optical Flow Computational Cost

**Issue**: Initial Farneback implementation with 50 frames and 5 pyramid levels took 18–25 seconds per video.

**Solution**:

- Reduced sample frames from 50 to 30 (40% reduction)
- Decreased pyramid levels from 5 to 3
- Used 0.5 pyramid scale instead of 0.7
- Frame skip strategy: sample every 2 seconds instead of every second

**Result**: Detection time reduced to 8–12 seconds with minimal accuracy loss (80% $\rightarrow$ 78%).

### 7.4 Problem 4: Edge Detection False Positives

**Issue**: Canny edge detection identified multiple rectangular objects (podiums, screens, doors) leading to 15% false positive rate.

**Solution**:

- Restricted detection to corner regions only
- Added size constraints (area between 0.8% and 25% of frame)
- Implemented interior darkness verification
- Prioritized contours with highest edge strength

**Result**: False positive rate reduced to 8%.

### 7.5 Problem 5: MediaPipe Small-Scale Detection

**Issue**: MediaPipe Pose struggled with small interpreter figures ($< 150 \times 150$ pixels), yielding confidence scores $< 0.3$.

**Solution**:

- Applied $2\times$ upscaling to corner ROIs before MediaPipe processing
- Lowered detection confidence from 0.7 to 0.5
- Used temporal averaging across 5 frames to stabilize detections

**Result**: Detection rate improved from 58% to 68% on small PiP overlays.

# 8 Experimental Validation

## 8.1 Dataset

- **Source**: 50 Sri Lankan parliamentary broadcast sessions
- **Duration**: 3–8 hours per session (total: 280 hours)
- **Resolution**: 1920×1080 at 30 FPS
- **PiP characteristics**: Bottom-right corner (92%), bottom-left (6%), top-right (2%)
- **Border types**: White (78%), cream (15%), light gray (5%), borderless (2%)

## 8.2 Ground Truth Annotation

Manual annotation of 500 randomly sampled frames:

- Bounding box coordinates
- Border presence/absence
- Border color category
- Interpreter visibility quality

## 8.3 Evaluation Metrics

$$\text{IoU} = \frac{|\text{Predicted} \cap \text{Ground Truth}|}{|\text{Predicted} \cup \text{Ground Truth}|} \tag{19}$$

$$\text{Detection} = \text{IoU} > 0.5 \tag{20}$$

$$\text{Accuracy} = \frac{\text{Correct Detections}}{\text{Total Frames}} \tag{21}$$

## 8.4 Results by Method

Table 8: Detection Method Comparison (500 Test Frames)

| **Method** | **Accuracy** | **Avg IoU** | **Time (s)** | **Usage %** |
|---|---|---|---|---|
| HSV Border | 92% | 0.87 | 3.2 | 78% |
| Optical Flow | 78% | 0.72 | 10.5 | 15% |
| Canny Edge | 71% | 0.65 | 5.8 | 5% |
| MediaPipe Pose | 68% | 0.68 | 12.3 | 2% |
| **Cascade** | **96%** | **0.89** | **4.1** | **100%** |

# 9 Conclusion

## 9.1 Key Achievements

1. Developed four complementary PiP detection methods with distinct strengths
2. Achieved 96% detection accuracy through intelligent cascade integration
3. Maintained computational efficiency (average 4.1 seconds per video)

4. Created robust system handling diverse broadcast conditions
5. Validated on 280 hours of real-world parliamentary footage

### 9.2 Method Selection Recommendations

- **Standard broadcasts with visible borders**: HSV Border Detection (fastest, most accurate)
- **Borderless or colored borders**: Optical Flow (motion-based robustness)
- **High background complexity**: Canny Edge or MediaPipe Pose
- **Production deployment**: Multi-method cascade (best overall performance)

### 9.3 Future Improvements

1. **Deep learning approach**: Train a YOLO or Faster R-CNN model for end-to-end PiP detection
2. **Temporal tracking**: Add Kalman filtering for smooth bounding box tracking across frames
3. **Adaptive thresholding**: Automatically tune HSV and motion thresholds per video
4. **Multi-interpreter detection**: Handle videos with multiple sign language interpreters
5. **Real-time processing**: Optimize for live broadcast detection ($< 100$ms latency)

### 9.4 Lessons Learned

- **No silver bullet**: Different broadcast conditions require different detection strategies
- **Cascade superiority**: Combining multiple methods outperforms any single approach
- **Domain-specific tuning**: Generic computer vision parameters require optimization for broadcast video
- **Validation importance**: Manual annotation of diverse test cases is essential for robust evaluation